19

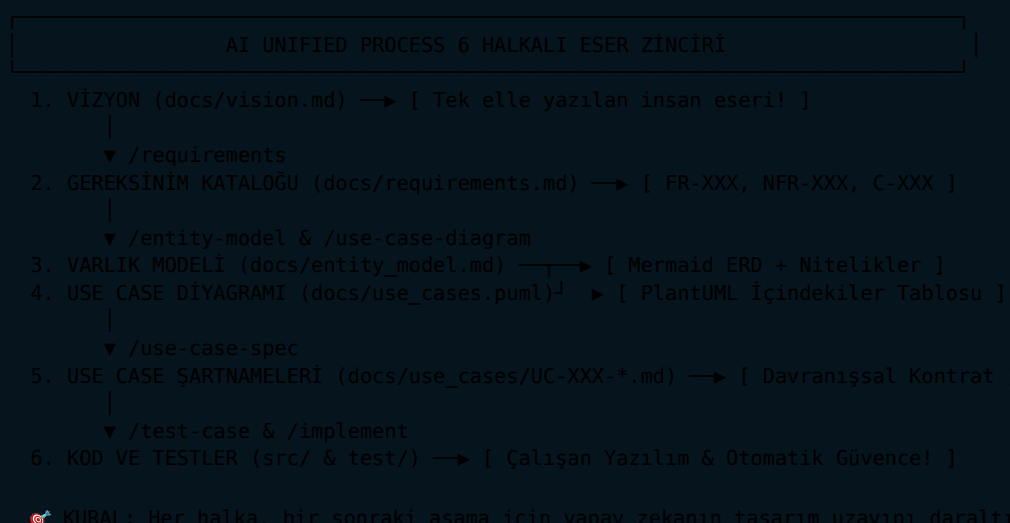
BÖLÜM: ESER ZINCIRI (ARTIFACT CHAIN) & PLANTUML MIMARISI | MODULE: THE ARTIFACT CHAIN & PLANTUML ARCHITECTURE

6 Halkalı Eser Zinciri: Vizyondan Koda Tasarım Uzayını Daraltma

The 6-Link Artifact Chain: Narrowing Design Space from Vision to Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir roket fırlatırken hedefe ulaşmak için 6 aşamalı bir kilit sistemi kullanırsın: 1) Haritada Mars'ı gösterirsin (Vizyon). 2) İhtiyaçları yazarsın: 'Oksijen, yakıt, 3 astronot' (Gereksinimler). 3) Roketin parçalarını çizersin (Varlık Modeli). 4) Görev adımlarını listelersin (Diyagram). 5) Her düğmenin ne zaman basılacağını kâğıda dökersin (Use Case). 6) Motorları ateşlersin (Kod!). Eğer direkt 1. adımdan 6. adıma atlarsan roket havada patlar! Zincirdeki her halka roketin rotasını milimetrik daraltır.

🇬🇧 ENGLISH (ELI5)

Launching an interplanetary rocket requires a six-stage telemetry sequence: 1) Pointing at Mars on the celestial map (Vision). 2) Sizing fuel and payload specifications (Requirements Catalog). 3) Blueprinting module geometries (Entity Model). 4) Indexing operational phases (Diagram). 5) Authoring step-by-step launch protocols (Use Case Specs). 6) Igniting the thrusters (Code & Tests). Skipping directly from Vision to Ignition ensures catastrophic atmospheric explosion. Each artifact progressively restricts variance until implementation becomes deterministic.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Eser Zinciri (The Artifact Chain) ve Tasarım Uzayı Mekaniği: Bir LLM'e tek satırlık bir prompt verip 'Bana bir e-ticaret sistemi yaz' dediğinizde tasarım uzayı (design space) sonsuzdur; model milyonlarca farklı mimari, tablo ve API rotası seçebilir ve bu da determinizmi sıfırlar. AI Unified Process bu sorunu 'Aşamalı Kısıtlama Zinciri' ile çözer: 1) docs/vision.md: Projede tek elle yazılan insan eseridir. Sistemin sınırlarını, aktörlerini ve alan adlarını belirler. 2) docs/requirements.md: Niyeti FR (Kullanıcı Hikayeleri), NFR ve Kısıtlar (Constraints) olarak kodlar. 3) docs/entity_model.md: Domain terimlerini ve ilişkilerini tek bir yerde tanımlar. 4) docs/use_cases.puml: Sistemin tüm use case haritasını çizer. 5) docs/use_cases/UC-XXX.md: Davranışı adım adım kilitler. `/implement` komutu çalıştığında yapay zekanın elinde vizyon, veri modeli, gereksinim satırı ve use case zaten mevcuttur; bu sayede üretilen kod tesadüfi değil, matematiksel bir kesinlik taşır. Hiçbir tekil dökümanın kusursuz olması gerekmez; bu katmanlı yapı sistemi sağlam kılar.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The mechanics of Design Space Constriction across the Artifact Chain: Prompting an LLM with 'Generate an e-commerce platform' presents an unconstrained, near-infinite degrees-of-freedom search space. The resulting architecture is chaotic, unprincipled, and un-reproducible. The AI Unified Process resolves this via a deterministic pipeline: each artifact progressively constricts the degrees of freedom for the next command. The vision document anchors boundary scope; the requirements catalog maps user stories to alphanumeric IDs; the entity model binds database nouns; the PlantUML diagram indexes the interaction portfolio; use case specifications define the step contracts. When `/implement` executes, the AI operates inside a tightly fenced sandbox. Reliable generation emerges from structural layering, not magical prompt engineering.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Eser Zincirinin Tamamını Yansıtan Temiz Mimari Yapısı
package com.maang.aiup.chain;

public class ArtifactChainDemonstration {
    // 1. docs/entity_model.md'den türeyen Domain Modeli
    public record BookCopyEntity(String copyId, String isbn, boolean isAvailable) {}

    // 2. docs/use_cases/UC-002-borrow-book.md'den türeyen Use Case Arayüzü
    public interface BorrowBookPort {
        void execute(String memberId, String copyId);
    }

    // 3. /implement tarafından üretilen temiz servis uygulaması
    public static class BorrowBookServiceImpl implements BorrowBookPort {
        @Override
        public void execute(String memberId, String copyId) {
            System.out.printf("Zincir tamamlandı: Üye %s -> Kopya %s başarıyla bağlandı.%n", memberId,
copyId);
        }
    }

    void main() {
        var service = new BorrowBookServiceImpl();
        service.execute("MEM-101", "CPY-99");
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Zincirdeki halkaları atlayıp doğrudan /use-case-spec veya /implement çalıştırmaya kalkmayın! Elinde bir varlık modeli (entity model) olmayan AI, her use case'de aynı tablo için farklı kolon isimleri uydurur (örn: bir yerde memberId, diğerinde user_id) ve veritabanı kilitlenir.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never short-circuit the artifact chain by executing `/implement` without an entity model. Deprived of a canonical entity dictionary, the AI will invent conflicting field names across specifications (e.g. `memberId` vs `user_id` vs `account_num`), triggering schema drift.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Eser zinciri 6 halkadan oluşur: Vizyon, Gereksinimler, Varlık Modeli, Diyagram, Şartnameler, Kod/Test.
- Tek elle yazılan döküman vizyondur; diğer tüm eserler komutlarla AI tarafından taslaklanır ve insan tarafından onaylanır.
- Her halka bir sonrakinin tasarım uzayını daraltarak rastgele kod üretimini deterministik kılar.

🎯 Key Architectural Takeaways

- The artifact chain spans 6 links: Vision, Requirements, Entity Model, Diagram, Specs, Code/Tests.
- Vision is the sole hand-written artifact; downstream artifacts are generated and human-refined.
- Each artifact narrows the design space, making AI code synthesis deterministic and predictable.